# 05 · 智能体 Harness 设计（具身版）

| 项 | 内容 |
|---|---|
| 文档 ID | TG-DOC-05 |
| 版本 | v1.0 |
| 编制日期 | 2026-10-25 |
| 依据 | Agent = Model + Harness；Diting-PdM Harness 十组件迁移 |
| 状态 | 待评审 |

---

## 1. 核心命题

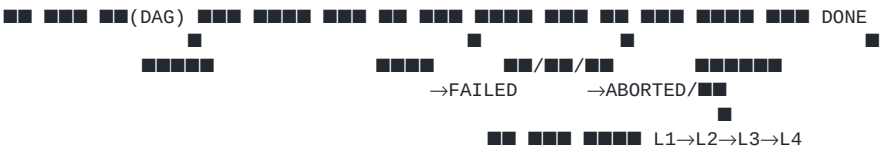
> Agent = Model + Harness。 模型再强，没有 Harness 也只是个聊天框。
> 具身场景尤甚：动作会伤人、会坏件、会说不清——Harness 不是增强项，是准入项。

天工 Harness 十组件（由 Diting-PdM 同构迁移，标注具身差异）：

| # | 组件 | Diting-PdM | 天工（具身） | 落点 |
|---|---|---|---|---|
| 1 | Agentic Loop | 显式状态机 | 同左 + 物理执行分支 | `orchestrator.py` |
| 2 | Tool Registry | 12 工具 | 5 类作业动作 + 只读工具 | `taskgraph.py` / `mcp.py` |
| 3 | Sandbox | 纯函数限额 | 安全审查（围栏/红线） | `safety.py` |
| 4 | Permission | allow/ask/deny | 同左 + 人工令牌簿 | `safety.py` / `server.py` |
| 5 | Memory | Hermes 三层 | episode 数据飞轮 | `episode.py` |
| 6 | Hooks | pre/post/stop | 同左 + 急停钩子 | `robots/base.py` |
| 7 | Sub-agents | 四角色会诊 | 规划/调度/安全/核验分离 | `orchestrator.py` |
| 8 | Skills | 受控沉淀 | 技能四态 + 灰度回滚 | `skills.py` |
| 9 | Observability | OTel 三层 span | 同左 + trace_id 贯穿 | `RunReport` |
| 10 | Eval Loop | golden cases 回放 | 同左 + 混沌注入 | `tests/` |

---

## 2. 编排闭环（组件 1）




关键实现（`orchestrator.py`）：

- 拓扑序遍历；依赖失败 → 下游 SUPERSEDED（不静默丢弃）
- 每个任务经 `_safe()` 安全审查后才下发链路层
- 停机 verdict（speed_limit=0）→ 任务 FAILED + `safety_halted` 标记

- 失败重试遵守状态机纪律：FAILED → 重新审批 → 再执行

---

## 3. 规划器（组件 2 · 大脑接口）

```
class Planner(Protocol):
    def plan(self, order: WorkOrder) -> Sequence[Task]: ...

RuleBasedPlanner()   # ■■■■■■■■■■■■■■■■■■
LLMPlanner()         # ■■■■■■■■■■■■■ DAG ■■
```

- LLM 规划器输出必须过同一 DAG 校验（环/重复 ID/悬空依赖一律拒绝）
- LLM 不可用 → 规则兜底并标记 `planner=rule_fallback`（FR-105）

---

## 4. 安全运行时（组件 3/4 · 脊髓）

```
check_action(action, zones, person_in_stop_zone)
  ■■ R1 ■■■■■■        → RedLineError■■■■■■■■■
  ■■ R2 ■■■■■■■      → ■■■■■■■■■■■■■■■
  ■■ R4 ■■■■■■        → Verdict(speed_limit=0) ■■
  ■■ ■■■■               → Verdict(speed_limit=z.speed_limit)
```

设计要点：安全判定在运行时层（Harness），不在提示词层。 提示词里的"禁止"不构成控制——这与 Diting-PdM 权限模型同构。

---

## 5. 降级阶梯（韧性）

| 级 | 动作 | 次数 | 需人 |
|---|---|---|---|
| L1 | 同版本重试 | 2 | 否 |
| L2 | 规则技能兜底 | 1 | 否 |
| L3 | 人工遥操作 | 1 | 是 |
| L4 | 挂起待人工 | — | 是 |

- 每次降级显式记录 `degraded=true` + 级数（禁止静默降级）
- 每迭代至少一次混沌验证（`--chaos` 演示模式即最小混沌测试）

---

## 6. 证据与可回放（组件 9）

每次运行产出 `RunReport`：

```
{
  "order_id": "WO-…", "trace_id": "TR-…",
  "degraded": false, "degrade_level": "NONE",
  "aborted": false,
  "tasks": {"T1": {"status": "DONE", "assigned_to": "AMR-01",
                   "evidence": ["trace:TR-…", "telemetry:sim://…", "skill_version:transport@1.0.0"]}},
  "events": [{"t": …, "what": "task_done", …}]
}
```

给定 trace_id 可重建：分解 → 审批 → 分配 → 安全裁定 → 执行 → 证据。

---

## 7. 技能治理（组件 8）

```
publish(PENDING) ■■approve■■■ APPROVED(■■)
                    ■
                    ■■■ ■■■■■■ ■■■ ■■■ SUPERSEDED
                    ■■■ rollback ■■■■ ■■■■■■■ APPROVED
```

- 执行只引用 APPROVED 版本；版本号写入证据链
- canary_pct 灰度比例随版本携带（现场分流）

---

## 8. 评测循环（组件 10）

- golden cases：分解结构、状态机、安全裁定、证据完整性的期望断言
- 混沌注入：`fail_next` 注入失败，验证降级阶梯逐级下降
- 三分离：Planner（生成计划）/ Executor（执行）/ Verifier（证据核验）互不信任